EWB
COURSES

EWB CLOUD LABS
STUDENT PROJECT REPORT

BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

AWS VPC Peering Between Two Amazon VPCs



2 VPCs / 2 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

NAME	**Deepak Yadav**
BATCH	**Batch 1**
PROJECT TITLE	**AWS VPC Peering Between Two Amazon VPCs**
AWS REGION	**US East (Ohio), us-east-2**
COMPLETION DATE	**21 October 2026**

01 / OVERVIEW

Project objective

Connect two VPCs and check that private communication works.

In this project I connected two Amazon VPCs in the US East (Ohio) Region (us-east-2) using a VPC peering connection, so that an EC2 instance in each VPC can talk to the other using the IPv4 address assigned to it inside its VPC. I set up the route tables and security groups for both sides and tested the connection from the command line with ping and SSH.

What VPC peering is

VPC peering is a private, one-to-one network link between two VPCs. Traffic between them stays on the AWS network and does not need an internet gateway. Creating the peering connection is not enough on its own. Each VPC also needs a route to the other VPC's CIDR block that points to the peering connection, and the security groups must allow the traffic. Peering is not transitive, and the two VPCs cannot have overlapping CIDR blocks.

What I did

- Created two VPCs with CIDR blocks that do not overlap: 28.0.0.0/16 (VPC1) and 28.200.0.0/16 (VPC2).
- Created the peering connection Peering-A-to-B (pcx-01d02da752e11444a), which shows Active.
- Added a route in each VPC that points to the other VPC through the peering connection.
- Set up security group rules for SSH and ICMP.
- Pinged in both directions using the instances' VPC IP addresses, and logged in by SSH to the VPC2 instance (which has no public IP) through the VPC1 instance.

About the evidence
Every value in this report comes from my screenshots, except where I say it comes from my own project notes. If a value is not visible in a screenshot, I have written that. Things the workbook asks for that I did not capture are marked 'Not captured'. Appendix B lists all of them.

02 / ARCHITECTURE

The network I built

Two VPCs, three EC2 instances and one peering connection.

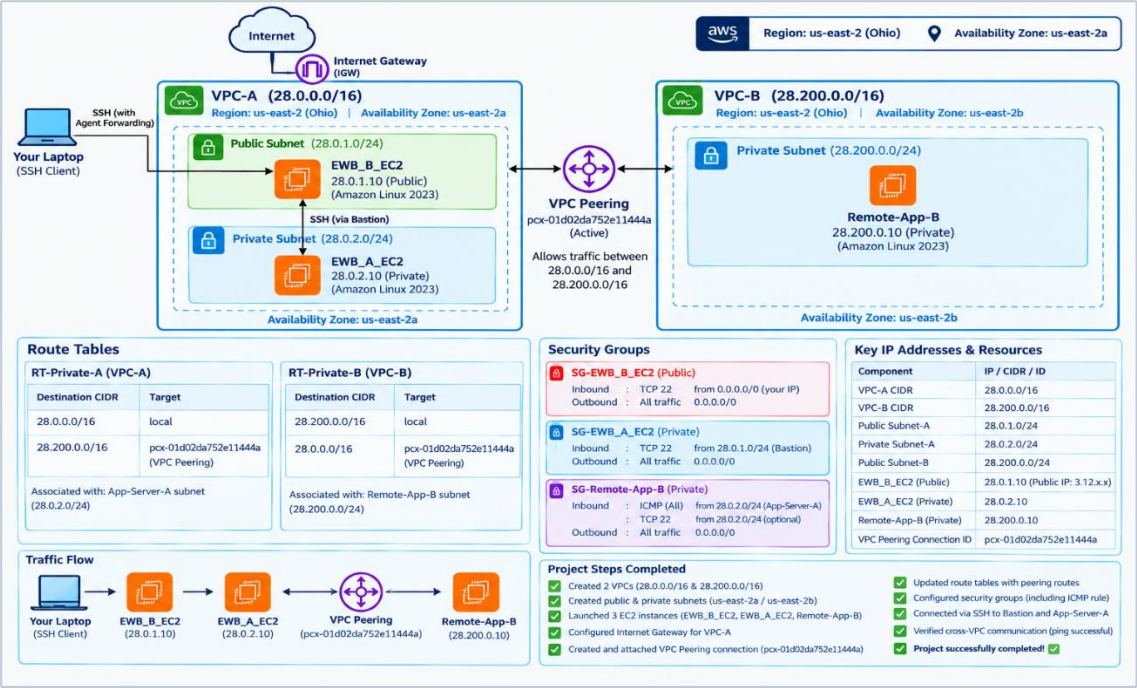


Figure 1. Architecture diagram drawn from my screenshots and notes (not a console screenshot).

Differences from the workbook design

The workbook expects a separate bastion host with a public IP and two application hosts with no public IP, plus three custom route tables and a private subnet in VPC A. I use three EC2 instances: EWB_B_EC2 (public IPv4, bastion/SSH entry point), EWB_A_EC2 (private IP 28.0.2.10), and Remote-App-B (private IP 28.200.0.10). It uses two VPCs connected through VPC Peering. The differences are:

Item	Workbook design	What I built
VPC CIDRs	10.10.0.0/16 and 10.20.0.0/16	28.0.0.0/16 and 28.200.0.0/16 (no overlap)
Region	Example: ap-south-1	us-east-2 (US East, Ohio)
EC2 instances	3: bastion, App-A, App-B	3: EWB_B_EC2 (Bastion) EWB_A_EC2 (App) Remote-App-B
Bastion	Separate bastion in the VPC A public subnet	EWB_B_EC2 is the SSH entry point.
Private subnet in VPC A	ewb-a-private with its own route table	Private-Subnet-A (28.0.2.0/24)
Route tables	3 custom tables with explicit associations	VPC-A: RT-Public-A (explicitly associated with the public subnet); the VPC-A subnet screenshots also show rtb-057b30af439246b18. VPC-B: main route table Is automatically associated with subnet that do not have an explicit route table association.
Instance type / OS	t3.micro suggested, Amazon Linux 2023	t3.micro, Amazon Linux 2023 (shown in the terminal banner)

03 / RESOURCE INVENTORY

Resources and IDs

Names, IDs and IP addresses used in this project.

Resource	Name	ID / value
VPC 1	VPC-A	vpc-0ac2a3fbc42c7db6e | 28.0.0.0/16
VPC 2	VPC-B	vpc-02f2414212ad6ccf3 | 28.200.0.0/16
VPC1 public subnet	Public-Subnet-A	subnet-0972b663595612d70 | 28.0.1.0/24 | us-east-2a
VPC1 private subnet	Private-Subnet-A	subnet-0bbbd325081d2641d | 28.0.2.0/24 | us-east-2a
VPC2 subnet	Private-Subnet-B	subnet-0a3570249720a6aeb | 28.200.1.0/16 | us-east-2a
Internet gateway	IGW-VPC-A	igw-060ef812d9974445a | target in the VPC1 public route table
VPC1 public route table	RT-Public-A	rtb-057b30af439246b18
VPC1 private route table	RT-Private-A	rtb-075f17b5cb3adc623
VPC2 route table	RT-Private-B	rtb-0fb58b6127fdd12ae
Peering connection	Peering-A-to-B	pcx-01d02da752e11444a | Active before cleanup
Security group, VPC1	SG-App-A SG-B	sg-0011d69026c7d79ac sg-0ea5e1593cb64dbf6
Security group, VPC2	SG-App-B	sg-0c8df501681dc2efb
EC2 instance, VPC1	EWB_A_EC2 EWB_B_EC2 (Bastion)	i-0818828d5eb7b8894 | private 28.0.2.10 | No public IPv4 i-0f13fe5c2e865a9ad | Private 28.0.1.10 | Public 3.17.81.47
EC2 instance, VPC2	Remote-App-B	i-05ac6ddb6ef813ee3 | private 28.200.0.10 | no public IPv4

All instances are in us-east-2a.

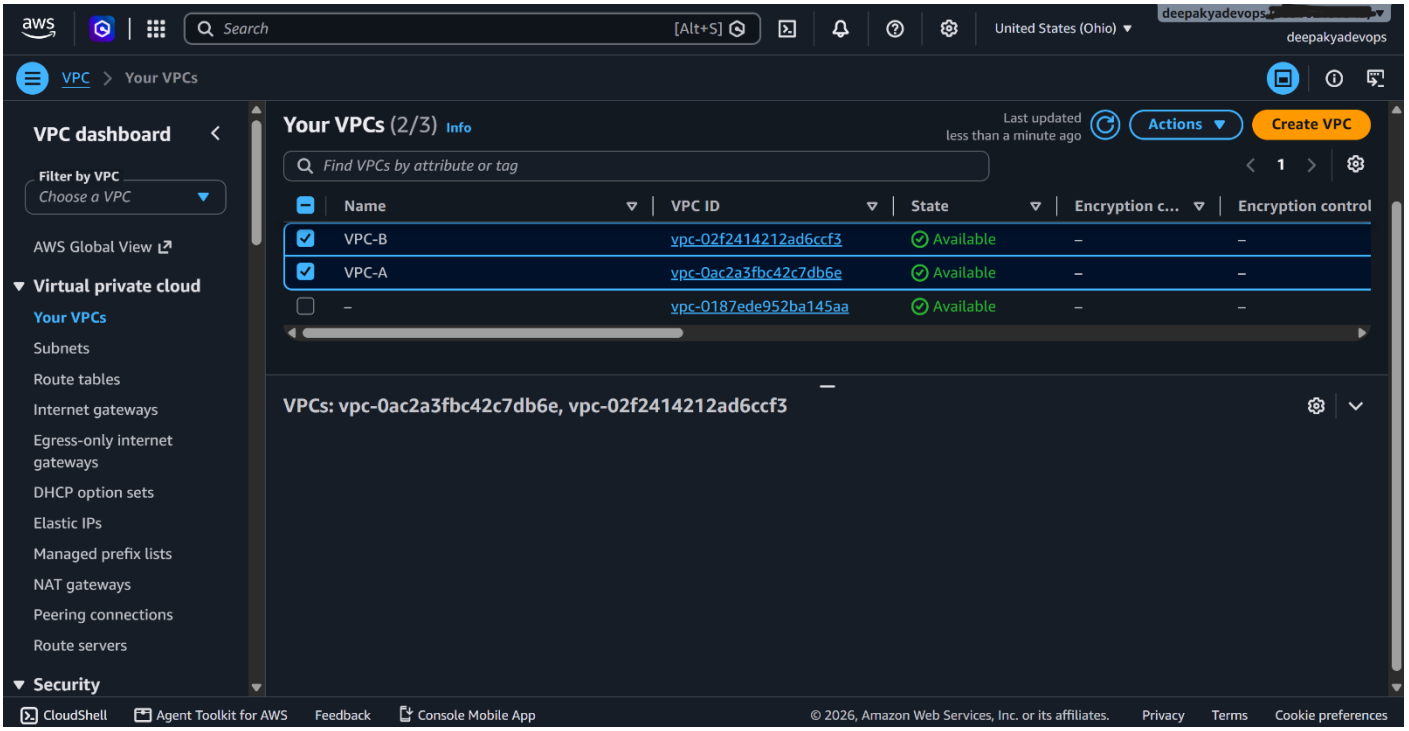


Figure 2. Your VPCs list: VPC-B (28.200.0.0/16) and VPC-A (28.0.0.0/16), both Available. The default VPC (172.31.0.0/16) is also listed.

04 / SUBNETS AND ROUTING

Subnets and route tables

The routes on both sides let traffic and replies use the peering connection.

Subnets

Subnet	ID	IPv4 CIDR	Auto-assign public IPv4	Route table
Public-Subnet-A (VPC-A)	subnet-0972b663595612d70	28.0.1.0/24	No	rtb-057b30af439246b18 | RT-Public-A
Private-Subnet-A (VPC-A)	subnet-0bbbd325081d2641d	28.0.2.0/24	No	rtb-075f17b5cb3adc623 | RT-Private-A
Private-Subnet-B (VPC-B)	subnet-0a3570249720a6aeb	28.200.0.0/16	Not captured	rtb-0fb58b6127fdd12ae | RT-Private-B

The two VPC-A subnets are in vpc-0ac2a3fbc42c7db6e, us-east-2a. My subnet screenshots were taken after I linked the public subnet to RT-Public-A, and a later screenshot shows that association.

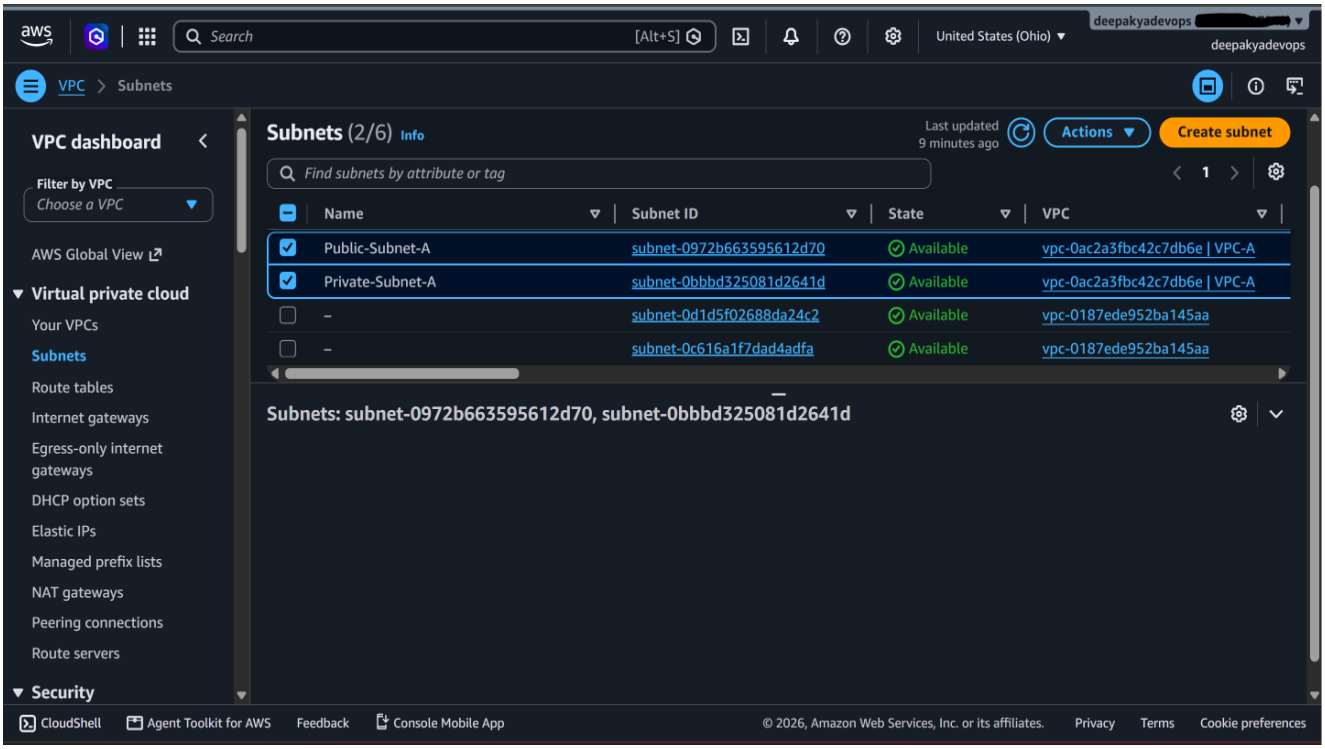


Figure 3. Subnets list taken before VPC-B was created: Public-Subnet-A (28.0.1.0/24) and Private-Subnet-A (28.0.2.0/24) in VPC-A.

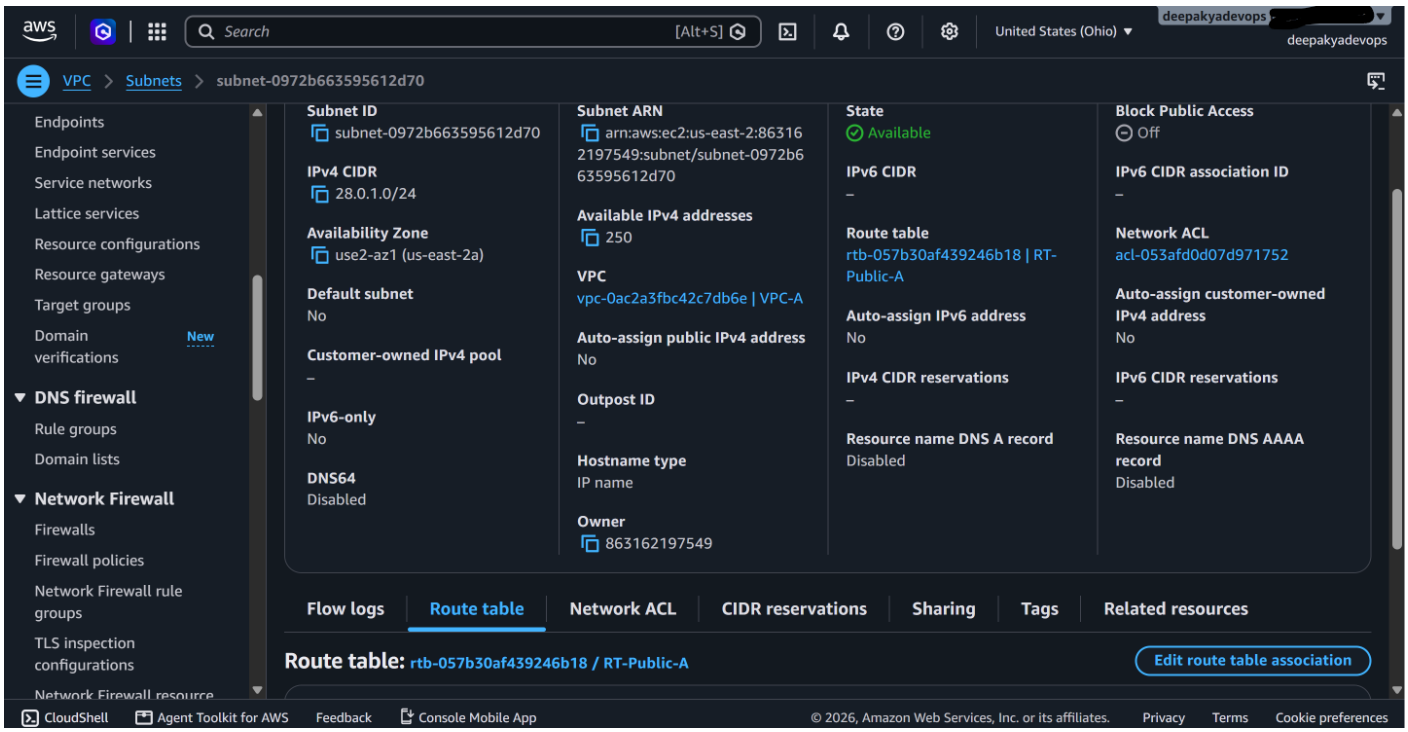


*Figure 4. Public-Subnet-A (subnet-0972b663595612d70): 28.0.1.0/24, us-east-2a, VPC-A, auto-assign public IPv4 No. The route table shown is **rtb-057b30af439246b18**.*

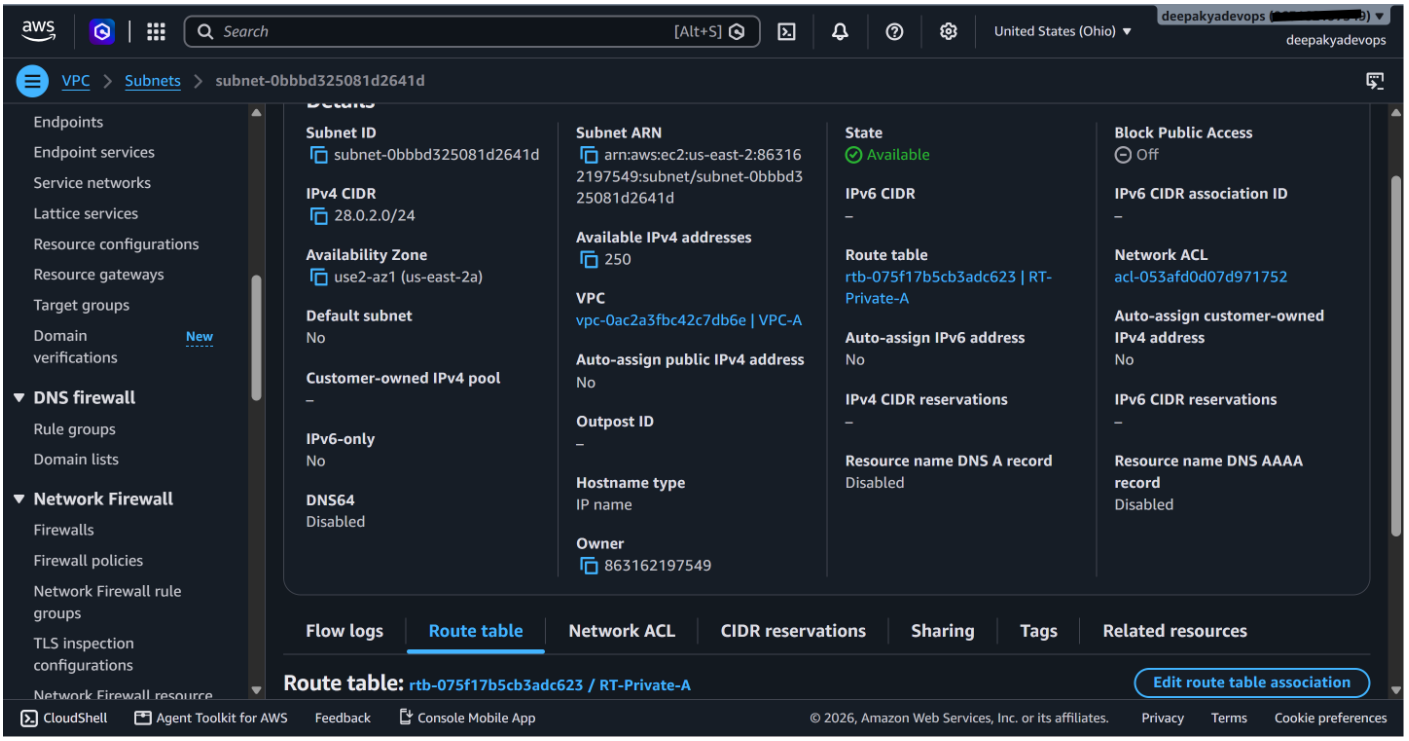


*Figure 5. Private-Subnet-A (subnet-0bbbd325081d2641d): 28.0.2.0/24, us-east-2a, VPC-A, auto-assign public IPv4 No. The route table shown is **rtb-075f17b5cb3adc623**.*

RT-Public-A (VPC-A) - rtb-057b30af439246b18

Destination	Target	Status	Purpose
0.0.0.0/0	igw-060ef812d9974445a	Active	Internet access (SSH from my laptop)
28.0.0.0/16	local	Active	Traffic inside VPC-A

This table is explicitly associated with subnet-0972b663595612d70 (Public-Subnet-A).

RT-Private-A (VPC-A) - rtb-075f17b5cb3adc623

Destination	Target	Status	Purpose
28.200.0.0/16	pcx-01d02da752e11444a	Active	Forward route to VPC-B through the peering connection.
28.0.0.0/16	local	Active	Traffic inside VPC-A

RT-Private-B (VPC-B)- rtb-0fb58b6127fdd12ae

Destination	Target	Status	Purpose
28.200.0.0/16	local	Active	Traffic inside VPC2
28.0.0.0/16	pcx-01d02da752e11444a	Active	Return route to VPC-A through the peering Connection. (VPC-B to VPC-A)

This is the main route table of VPC-B. The route destinations match the VPC CIDRs: 28.200.0.0/16 on the VPC-A side and 28.0.0.0/16 on the VPC-B side. Because both sides have a route, replies can find their way back.

Route table before the peering route

Figure 6 shows RT-Public-A after the Internet Gateway route was added but before the VPC peering route was configured. The route table contains two active routes: the local route (28.0.0.0/16), which enables communication within VPC-A, and the internet route (0.0.0.0/0), which directs internet bound traffic to the internet Gateway. At this stage, there is no route to VPC-B (28.200.0.0/16) through the VPC peering connection. The route table is explicitly associated with Public-Subnet-A. This screenshot documents the route table's configuration before the peering route was added.

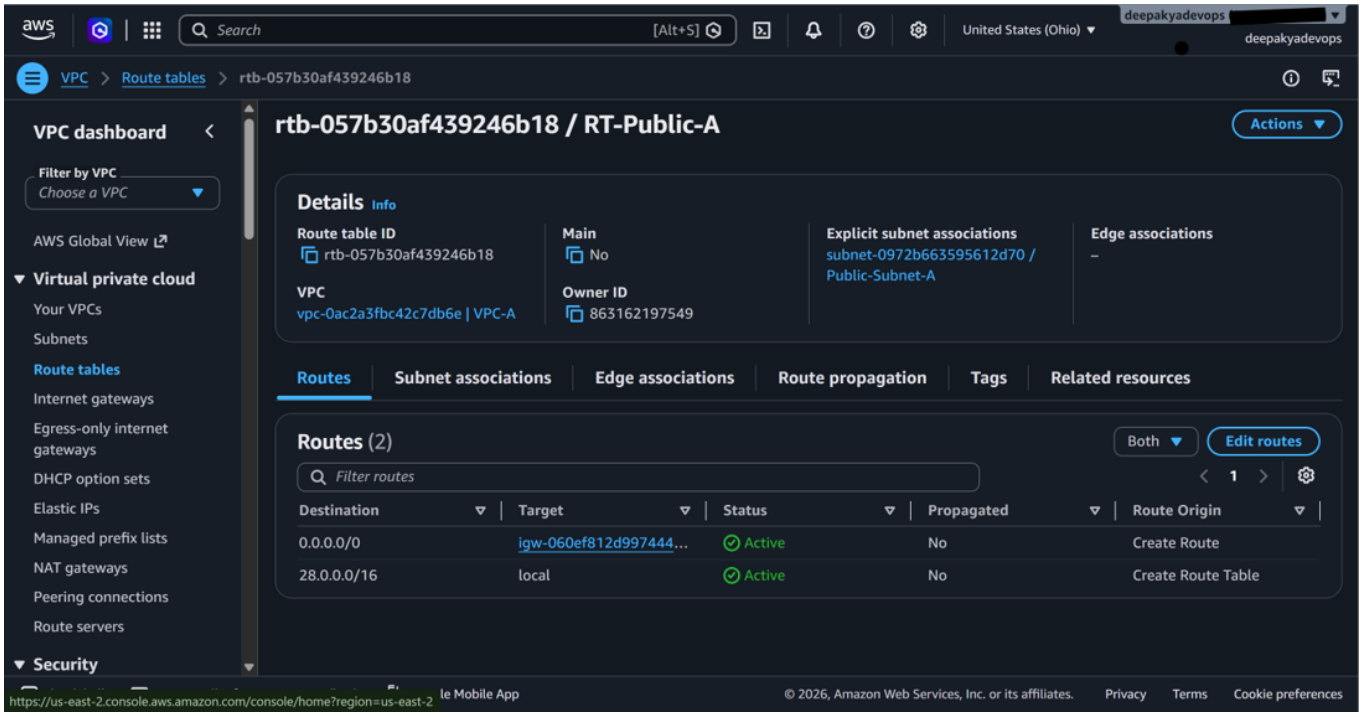


Figure 6. RT-Public-A right after I added the internet route: only 0.0.0.0/0 to igw-060ef812d9974445a... and 28.0.0.0/16 local. No route to 28.200.0.0/16.

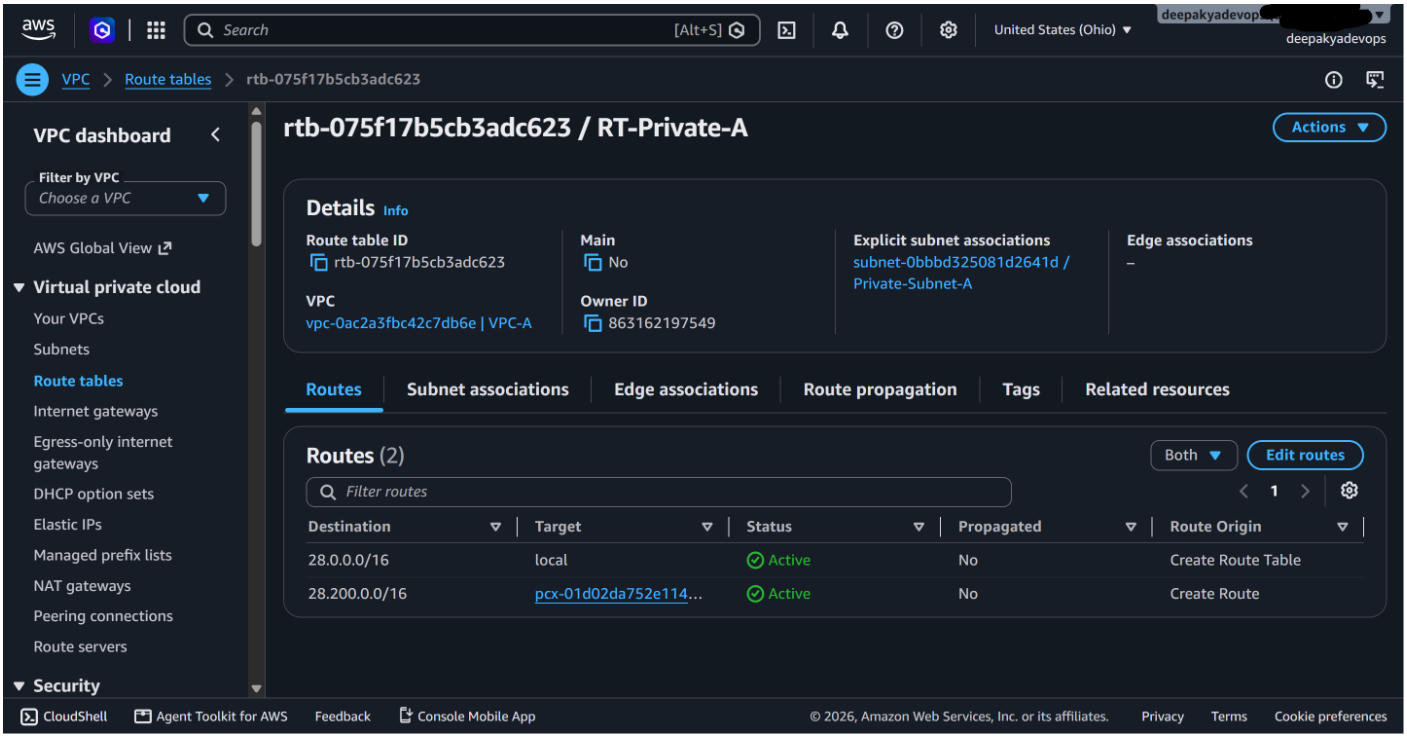


Figure 7. VPC-A, RT-Private-A (rtb-075f17b5cb3adc623).

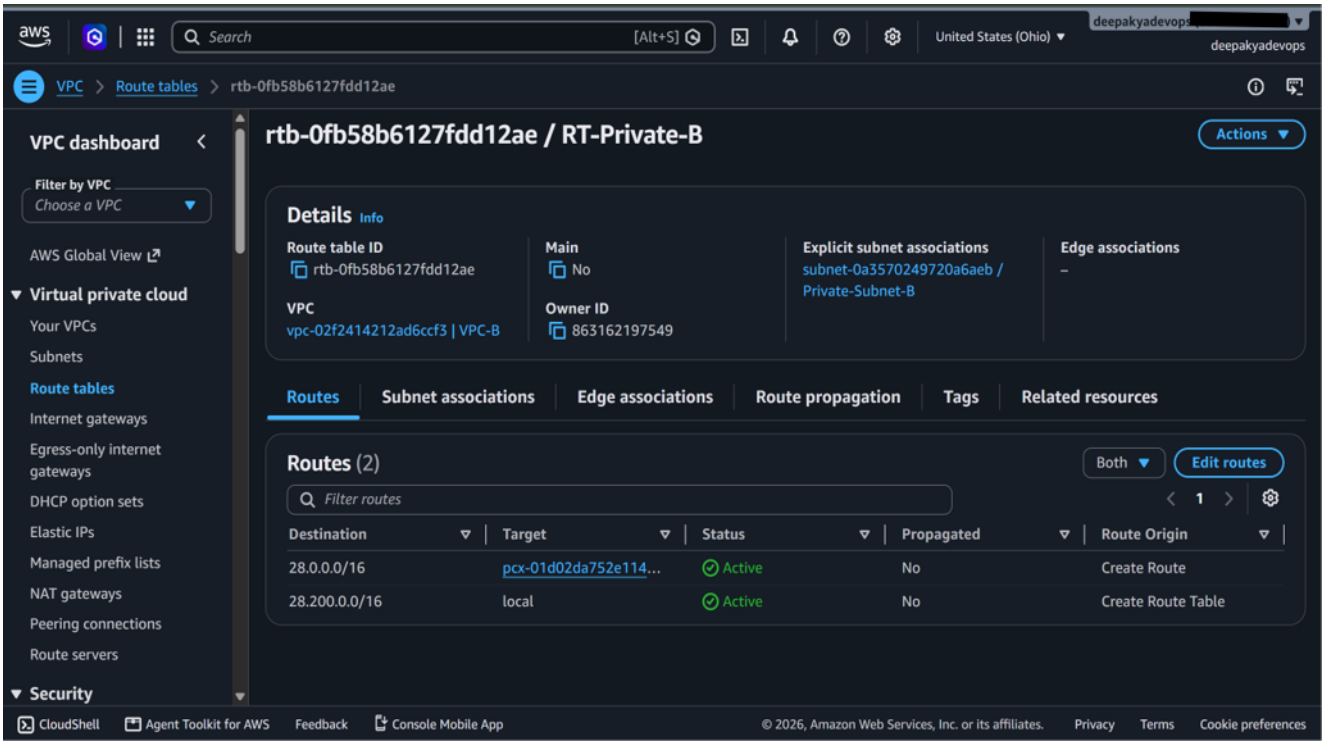


Figure 8. VPC-B route table rtb-0fb58b6127fdd12ae with the return route 28.0.0.0/16 to pcx-01d02da752e11444a.

05 / PEERING CONNECTION

Request, accept, activate

The connection is the link. Routes and security groups decide what can use it.

Field	Value (from Figure 9)
Name / ID	Peering-A-to-B / *pcx-01d02da752e11444a*
Status	Active (expiration time is blank)
Requester VPC / CIDR	vpc-0ac2a3fbc42c7db6e (VPC-A) / 28.0.0.0/16
Accepter VPC / CIDR	vpc-02f2414212ad6ccf3 (VPC-B) / 28.200.0.0/16
Requester and accepter Region	Ohio (us-east-2) for both
Requester and accepter owner	Same AWS account (both owner IDs are the same)
DNS resolution over peering	Disabled in both directions. My tests used IP addresses, so I did not need it.

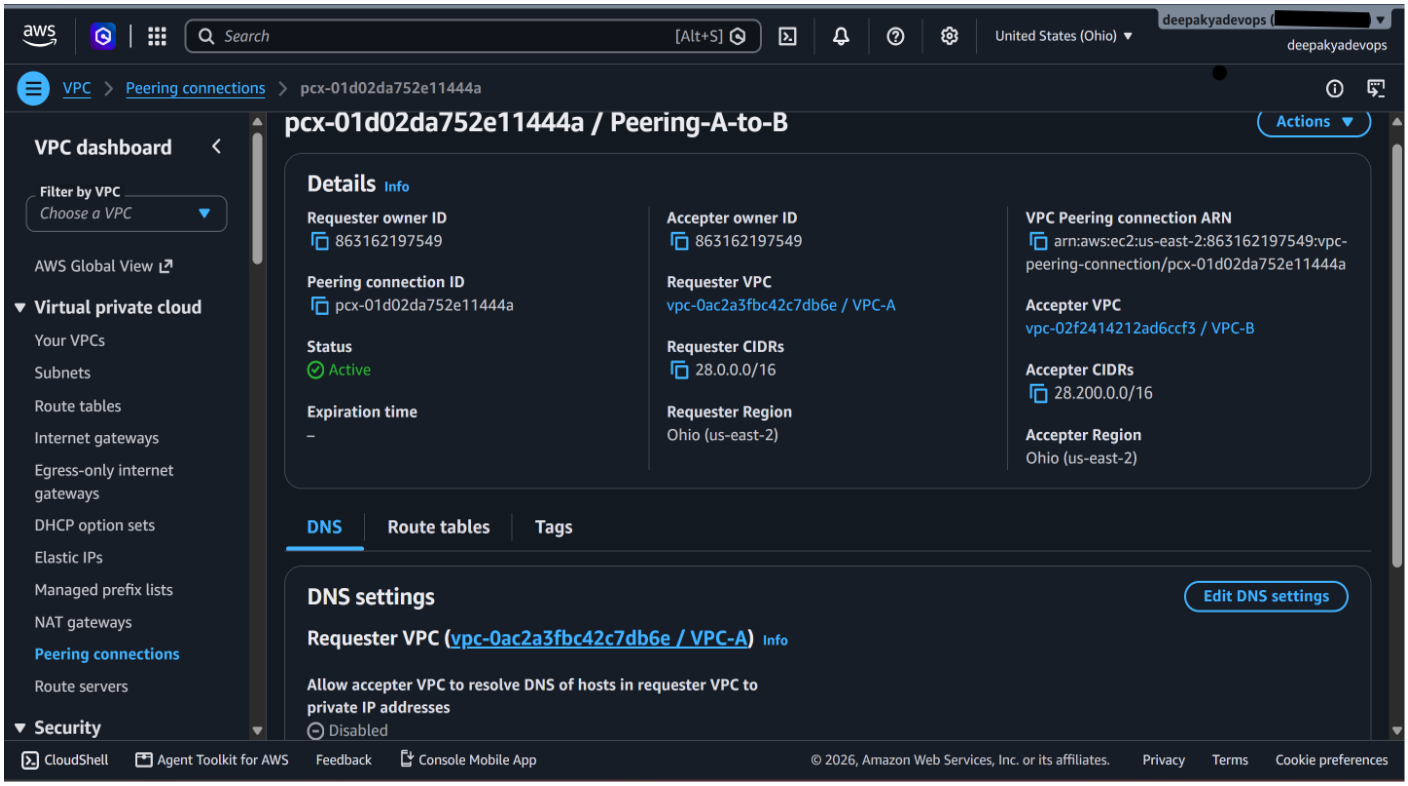


Figure 9. Peering connection pcx-01d02da752e11444a / Peering-A-to-B: Status Active, requester VPC-A (28.0.0.0/16), accepter VPC-B (28.200.0.0/16), both in us-east-2.

Inbound rules

Only the traffic needed for SSH and ping.

sg-0ea5e1593cb64dbf6 - SG-B (VPC-A)

Rule ID	Type	Protocol / port	Source
sgr-04295d2b9d0f7c86b	SSH	TCP / 22	103.177.203.5/32

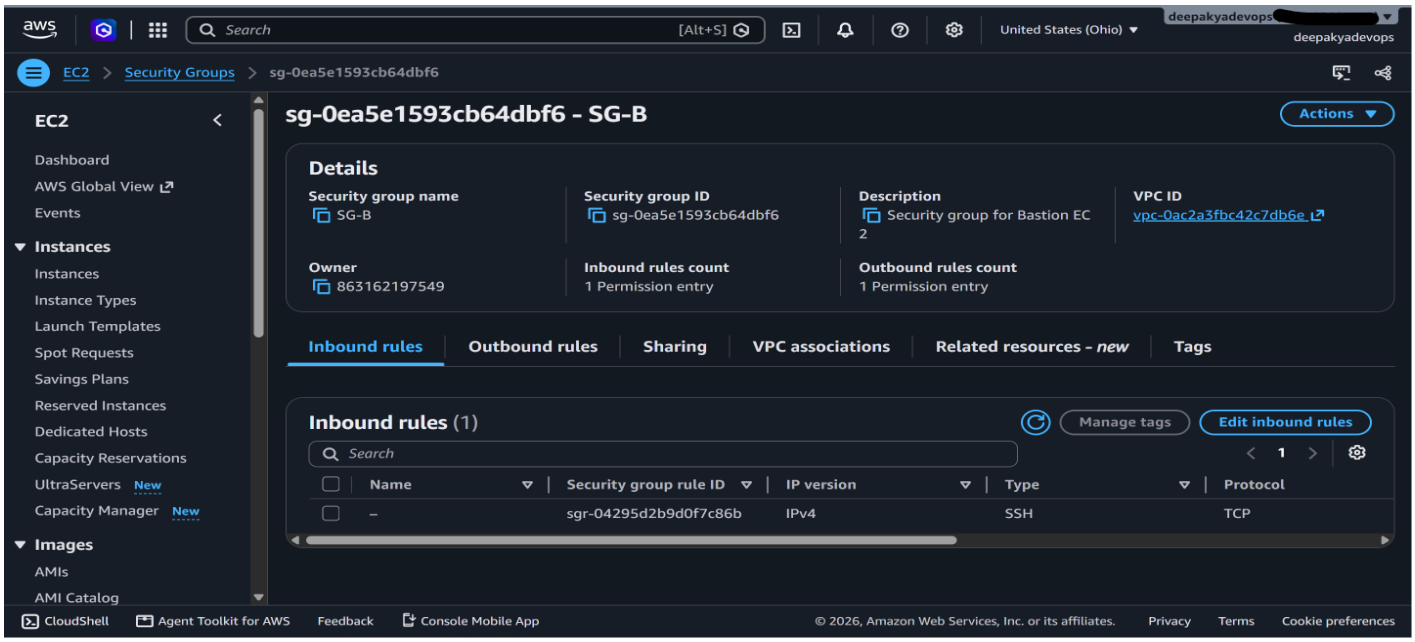


Figure 10. SG-B (sg-0ea5e1593cb64dbf6): One inbound rule (SSH)

sg-0011d69026c7d79ac – SG-App-A

Rule ID	Type	Protocol / port	Source
sgr-0f942b8abc5e8d379	SSH	TCP / 22	SG-B (VPC-A)

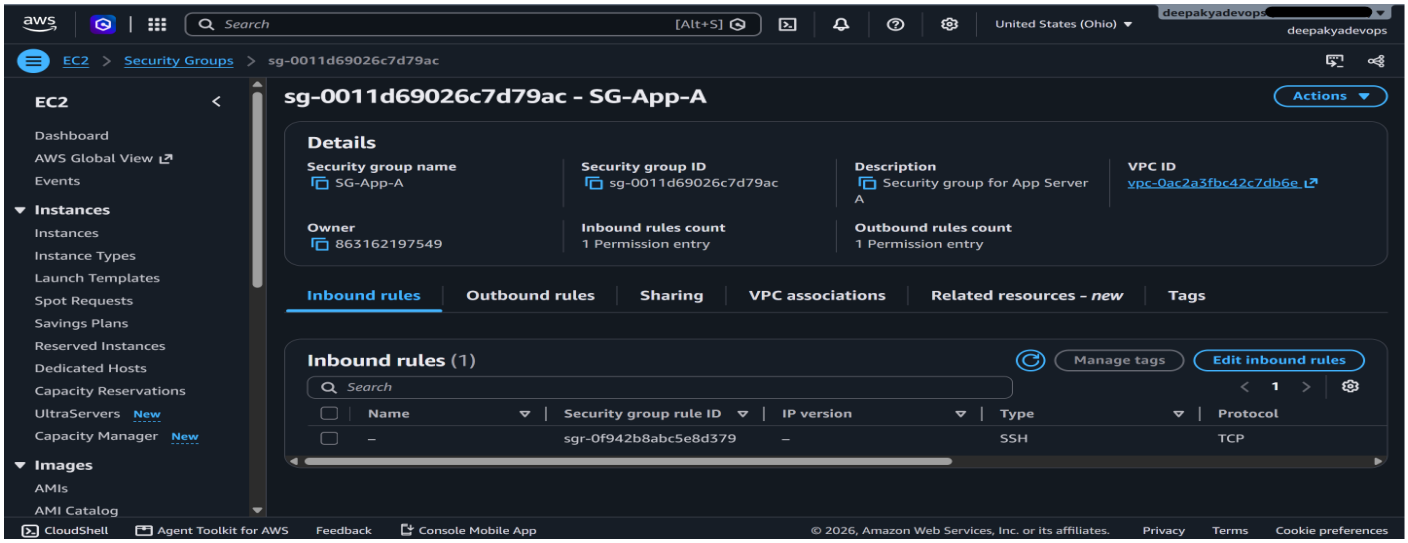


Figure 11. SG-App-A (sg-0011d69026c7d79ac): One inbound rule (SSH)

sg-0faec46e705c1ec19 – SG-App-B (VPC-B)

Rule ID	*Type*	*Protocol / port*	*Source*
sgr-04aaabdf32359ff26	*SSH*	*TCP / 22*	*28.200.0.0/16 (VPC-B)*
sgr-0bd584e317bf4c2d3	*SSH*	*TCP / 22*	*28.0.2.0/24*
sgr-0506381a2a948900a	*All ICMP – IPv4*	*ICMP / All*	*28.0.2.0/24*

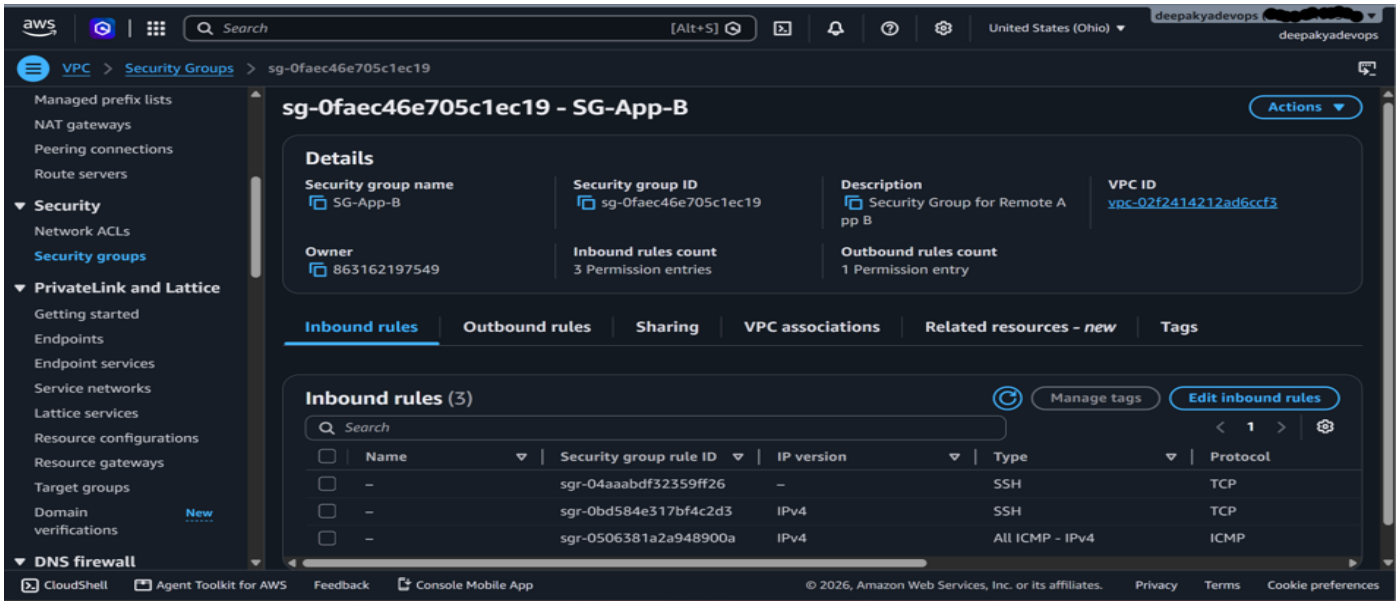


Figure 12. SG-App-B (sg-0faec46e705c1ec19): three inbound rules (One ICMP and two SSH).

07 / COMPUTE

EC2 instances

Three Amazon Linux 2023 instances.

Field	EWB_A_EC2	EWB_B_EC2 (BASTION)	Remote-App-B
Instance ID	i-0818828d5eb7b8894	i-0f13fe5c2e865a9ad	i-05ac6ddb6ef813ee3
State / status checks	Running / 3/3 passed	Running / 3/3 passed	Running / 3/3 passed
Type / zone	t3.micro / us-east-2a	t3.micro / us-east-2a	t3.micro / us-east-2a
Private IPv4	28.0.2.10	28.0.1.10	28.200.0.10
Public IPv4	None (field empty)	3.17.81.47	None (field empty)
Security group	SG-App-A	SG-B	SG-App-B

The Public IPv4 field for EWB_VPC2_EC2 is empty, so that instance has no direct internet address. This is only one part of the workbook design: the workbook also expects a separate bastion and two private application hosts, and my setup has a public IPv4 address on the VPC1 instance and only two instances (see Section 2).

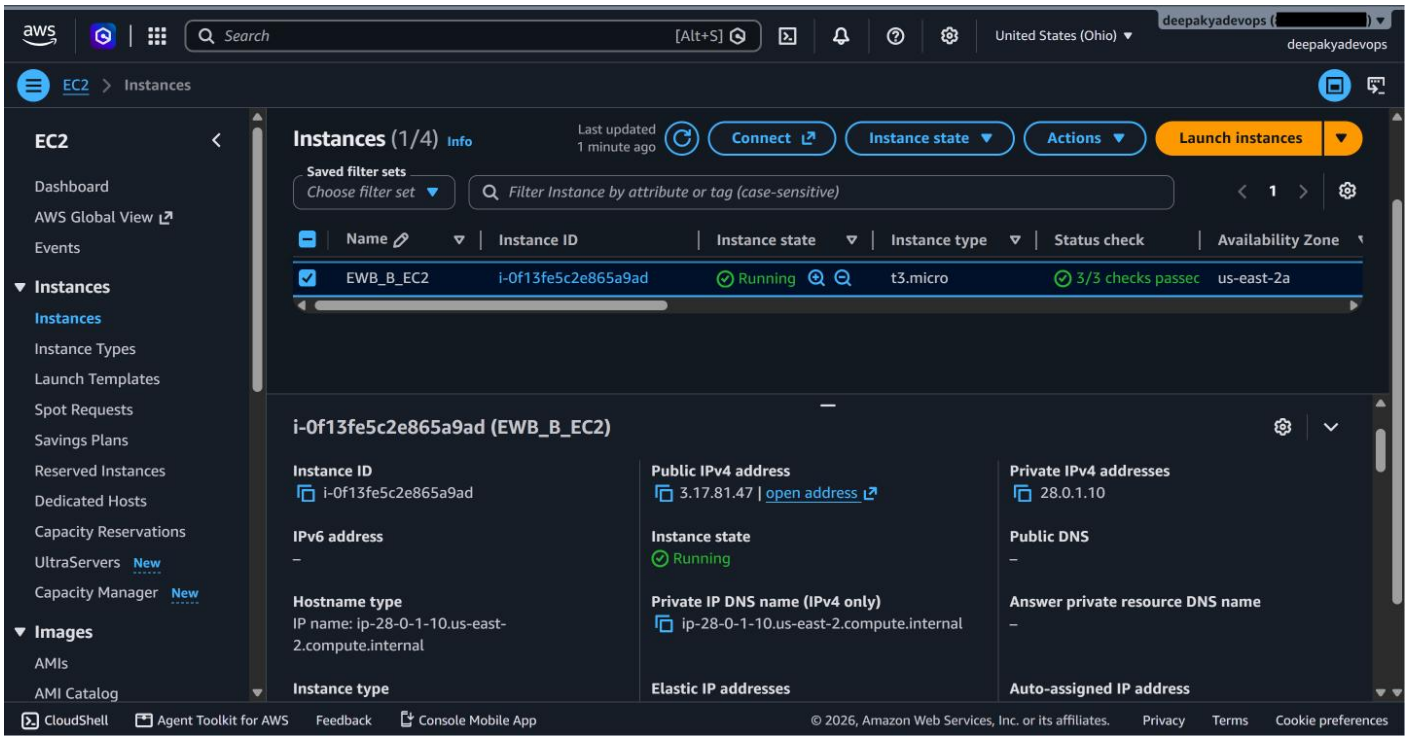


Figure 13. EWB_B_EC2 {BASTION} (i-0f13fe5c2e865a9ad): Running, t3.micro, us-east-2a, private IPv4 28.0.1.10, public IPv4 3.17.81.47.

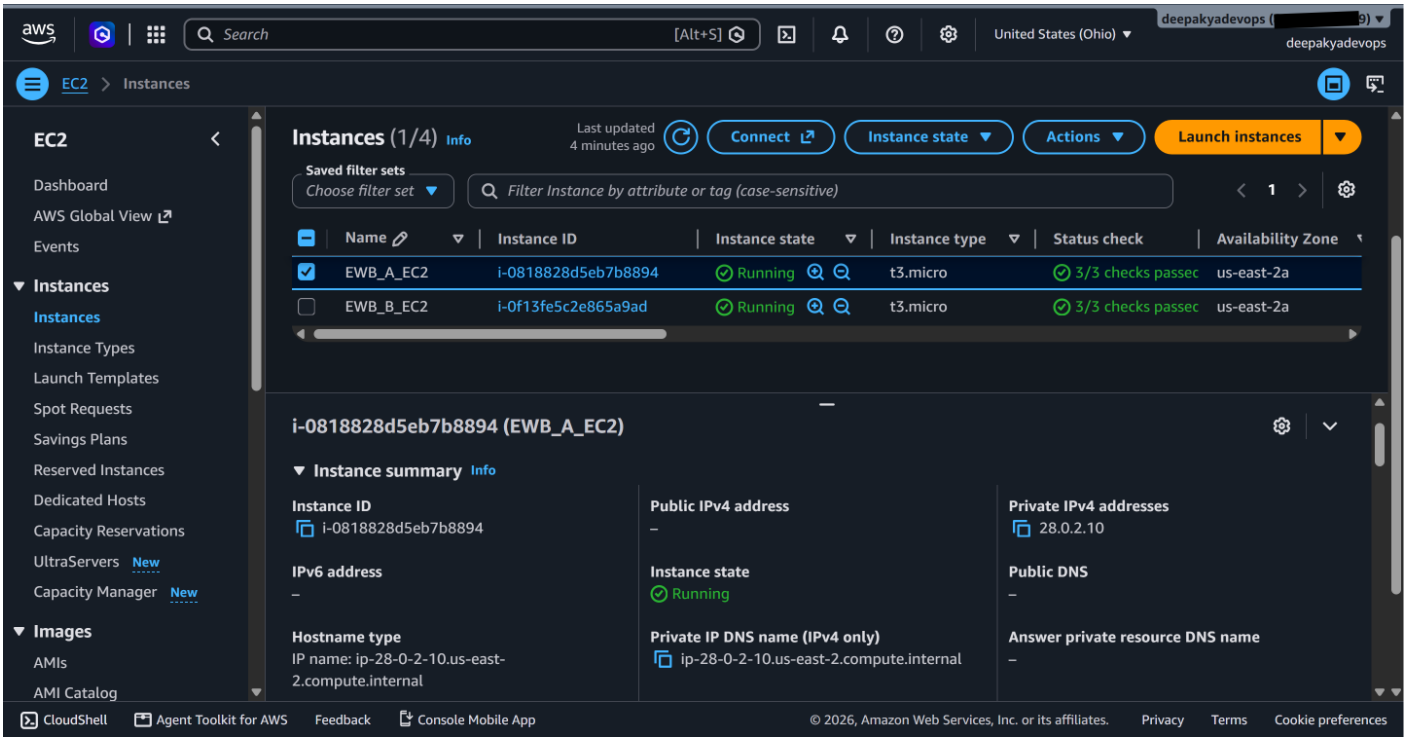


Figure 14. EWB_A_EC2 (i-0818828d5eb7b8894): Running, t3.micro, us-east-2a, private IPv4 28.0.2.10, Public IPv4 address empty.

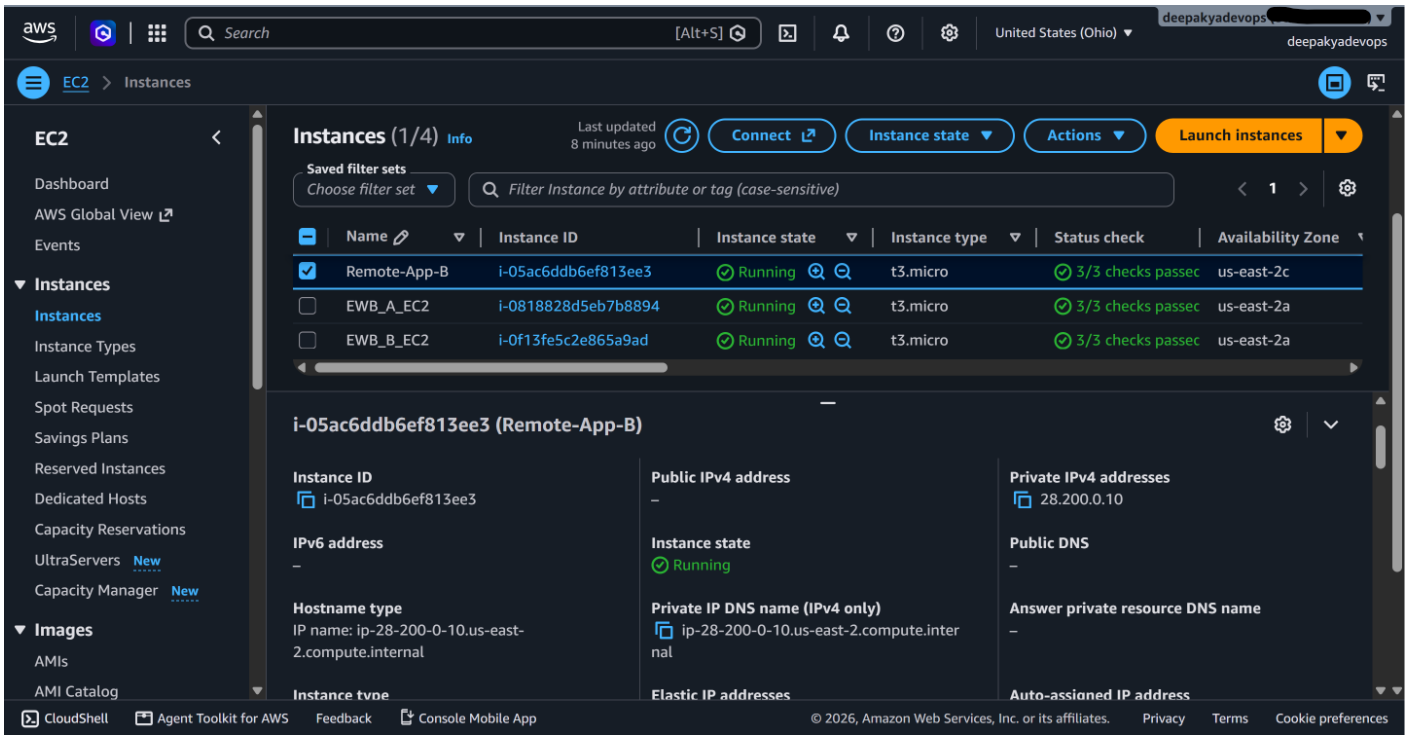


Figure 15. Remote-App-B(i-05ac6ddb6ef813ee3): Running, t3.micro, us-east-2c, Private IPv4 28.200.0.10, Public IPv4 is empty.

08 / VERIFICATION

Connectivity tests

Results from project testing and available terminal records.

Test	Where / command	Result	Figure
TCP 22 to Bastion	Laptop: Windows PowerShell 3.17.81.47 : 22	SSH connectivity successful	Fig 16
SSH to Bastion	Laptop: ssh -i key.pem ec2-user@3.17.81.47	Login successful (Amazon Linux 2023)	Fig 16
Forward ping	EWB_A_EC2: ping -c 4 28.200.0.10	Successful 4 sent, 4 received, 0% loss	Fig 18
Private SSH to App Server A	Laptop: ssh ec2-user@28.0.2.10 from EWB_B_EC2	Login successful: banner shows last login from 103.177.203.5	Fig 16
Reverse ping	Remote-App-B: ping -c 4 28.0.2.10	Successful 4 sent, 4 received, 0% loss	No screenshot taken at that time.
Isolation	EC2 instance details, Remote-App-B	Public IPv4 address is empty; instance is configured without direct public IPv4 access.	-
Baseline (before peering)	Ping between VPC-A and VPC-B before peering routes Were configured.	Not captured as a test. I only have the route table screenshot from before the peering route was added (a configuration screenshot, not a result).	-

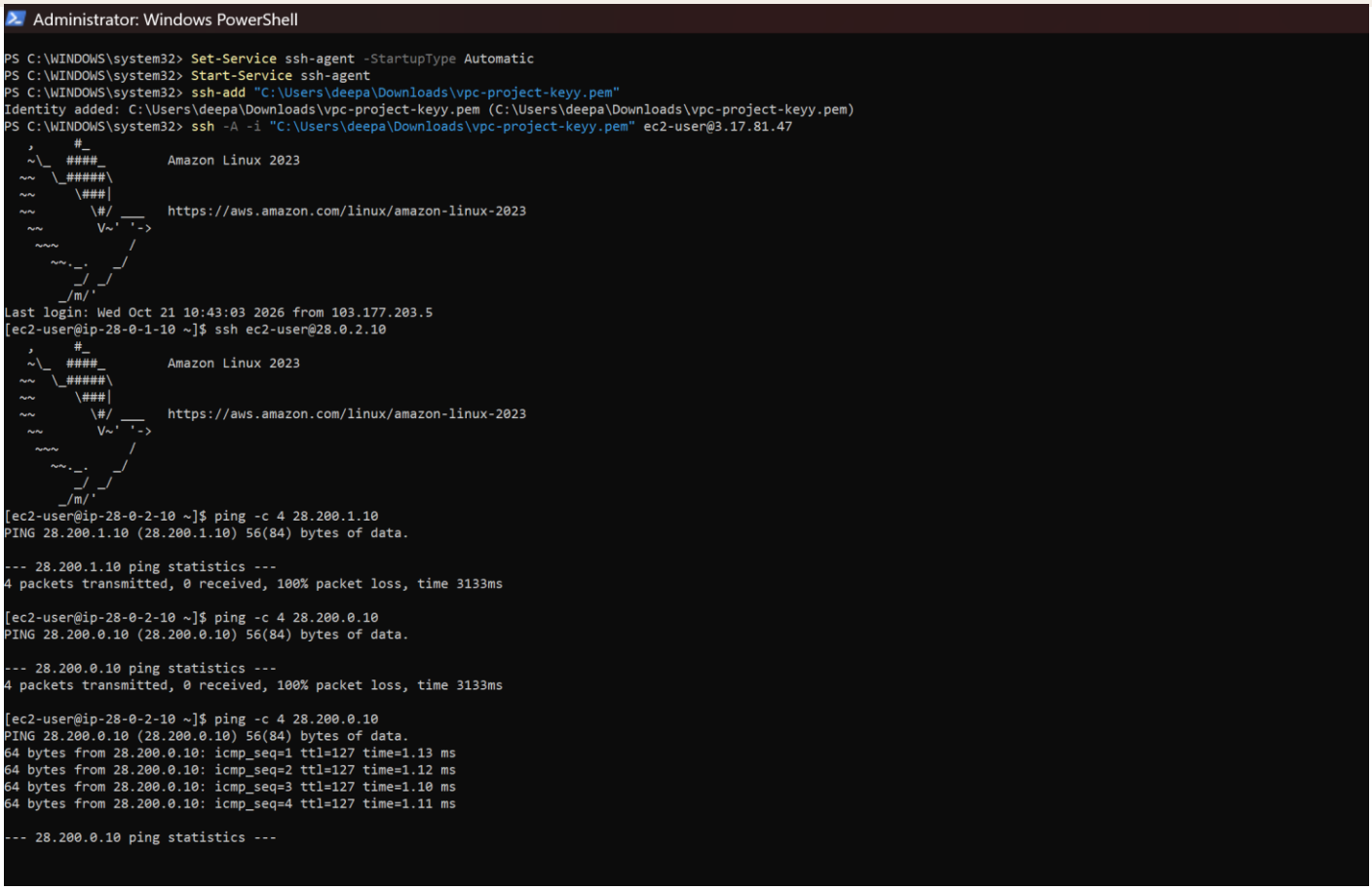

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Set-Service ssh-agent -StartupType Automatic
PS C:\WINDOWS\system32> Start-Service ssh-agent
PS C:\WINDOWS\system32> ssh-add "C:\Users\deepa\Downloads\vpc-project-keyy.pem"
Identity added: C:\Users\deepa\Downloads\vpc-project-keyy.pem (C:\Users\deepa\Downloads\vpc-project-keyy.pem)
PS C:\WINDOWS\system32> ssh -A -i "C:\Users\deepa\Downloads\vpc-project-keyy.pem" ec2-user@3.17.81.47
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Wed Oct 21 10:43:03 2026 from 103.177.203.5
[ec2-user@ip-28-0-1-10 ~]$ ssh ec2-user@28.0.2.10
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
[ec2-user@ip-28-0-2-10 ~]$ ping -c 4 28.200.1.10
PING 28.200.1.10 (28.200.1.10) 56(84) bytes of data.
--- 28.200.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3133ms
[ec2-user@ip-28-0-2-10 ~]$ ping -c 4 28.200.0.10
PING 28.200.0.10 (28.200.0.10) 56(84) bytes of data.
--- 28.200.0.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3133ms
[ec2-user@ip-28-0-2-10 ~]$ ping -c 4 28.200.0.10
PING 28.200.0.10 (28.200.0.10) 56(84) bytes of data.
64 bytes from 28.200.0.10: icmp_seq=1 ttl=127 time=1.13 ms
64 bytes from 28.200.0.10: icmp_seq=2 ttl=127 time=1.12 ms
64 bytes from 28.200.0.10: icmp_seq=3 ttl=127 time=1.10 ms
64 bytes from 28.200.0.10: icmp_seq=4 ttl=127 time=1.11 ms
--- 28.200.0.10 ping statistics ---

Figure 16.

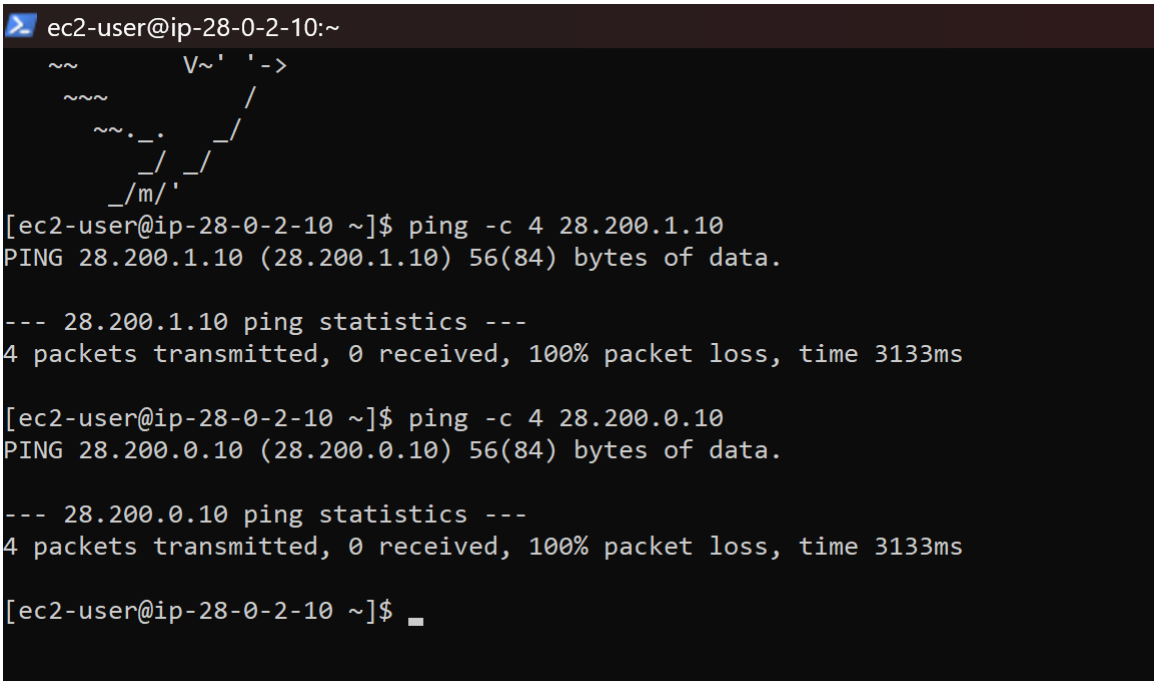


Figure 17.

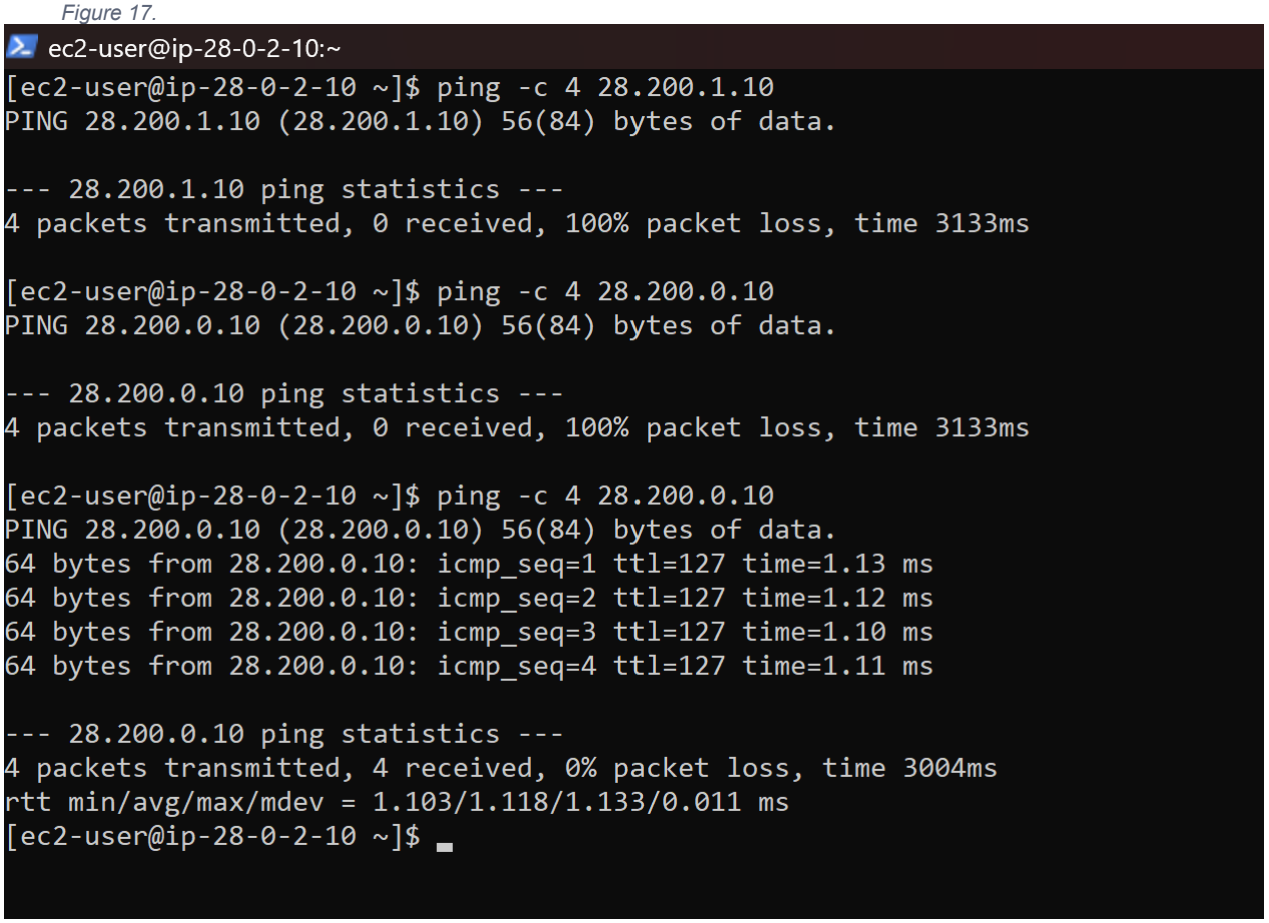


Figure 18. failure.

09 / FAILURE AND RECOVERY

ICMP failure and recovery

What my screenshots show and what they do not.

Step	Command	Result	Figure
Failure	ping -c 4 28.200.1.10	4 sent, 0 received, 100% packet loss	Fig 18
Recovery	ping -c 4 28.200.1.10	4 sent, 4 received, 0% loss	

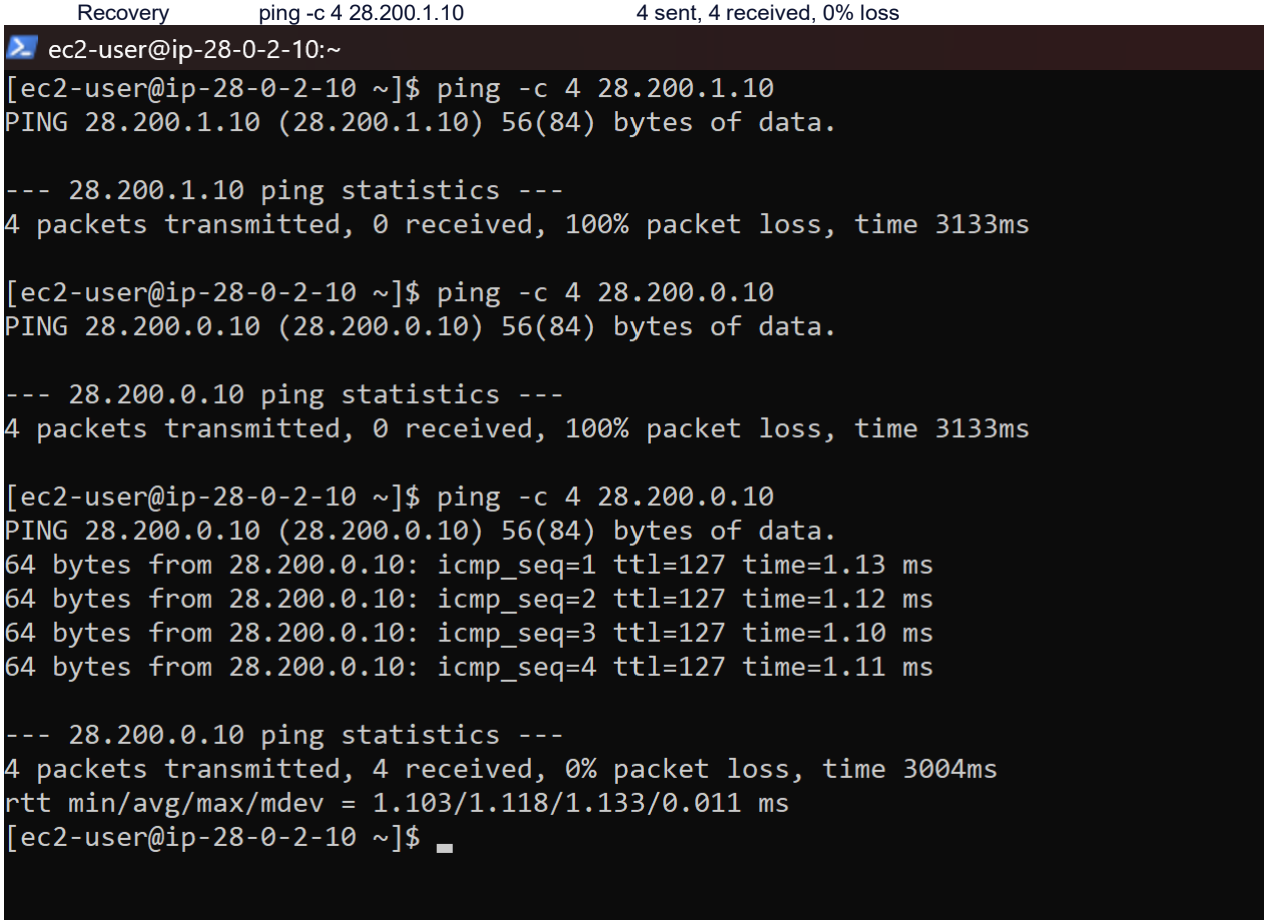


Figure 18.

10 / TRAFFIC PATHS

Forward path and return path

Why both routes and both security rules are needed.

Forward path: EWB_B_EC2 to Remote-App-B

1. The destination 28.200.1.179 matches 28.200.0.0/16 in RT-Public-A, whose target is / *pcx-01d02da752e11444a* . This route is more specific than 0.0.0.0/0, so the internet gateway route is not used.
2. The peering connection delivers the packet into VPC2-B.

Return path: Remote-App-B back to EWB_B_EC2

1. Security groups are stateful, so replies to allowed traffic are allowed automatically.

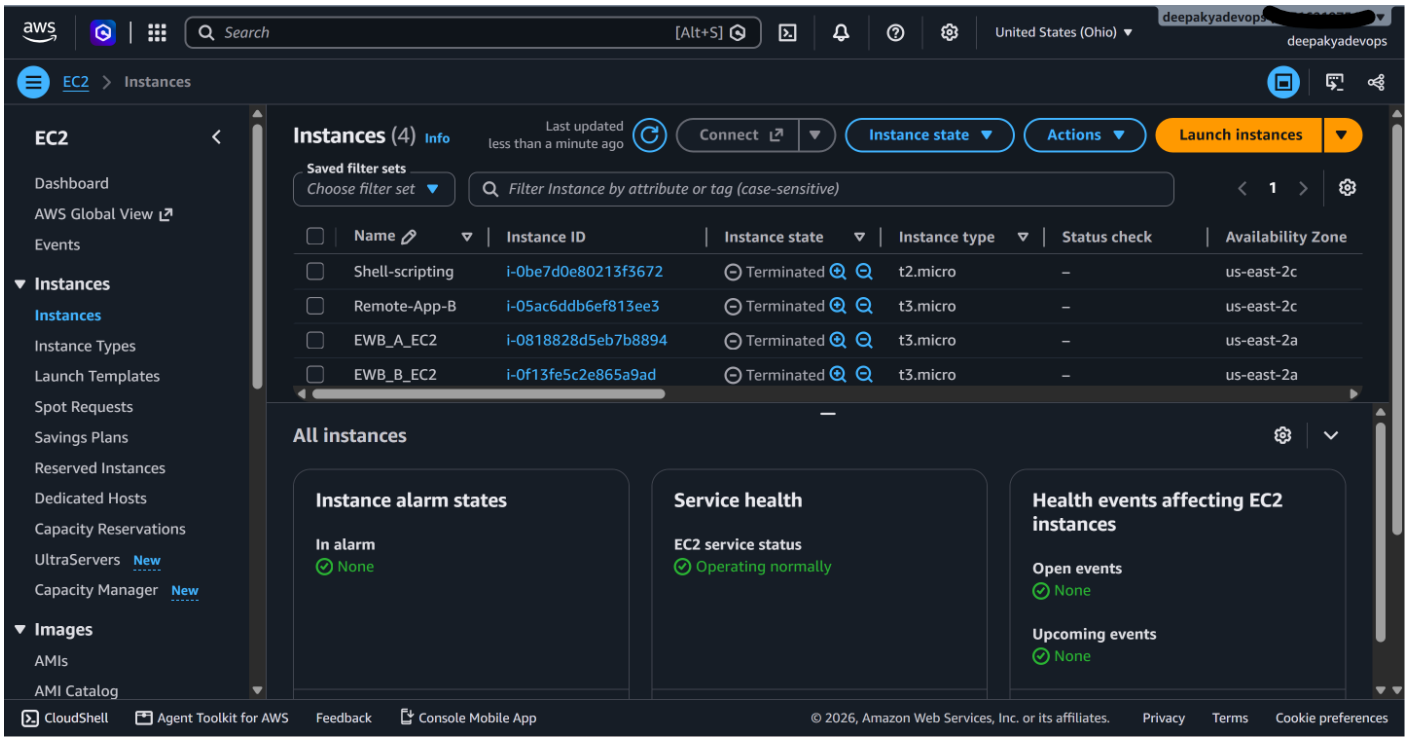


Figure 19. Cleanup, instances.

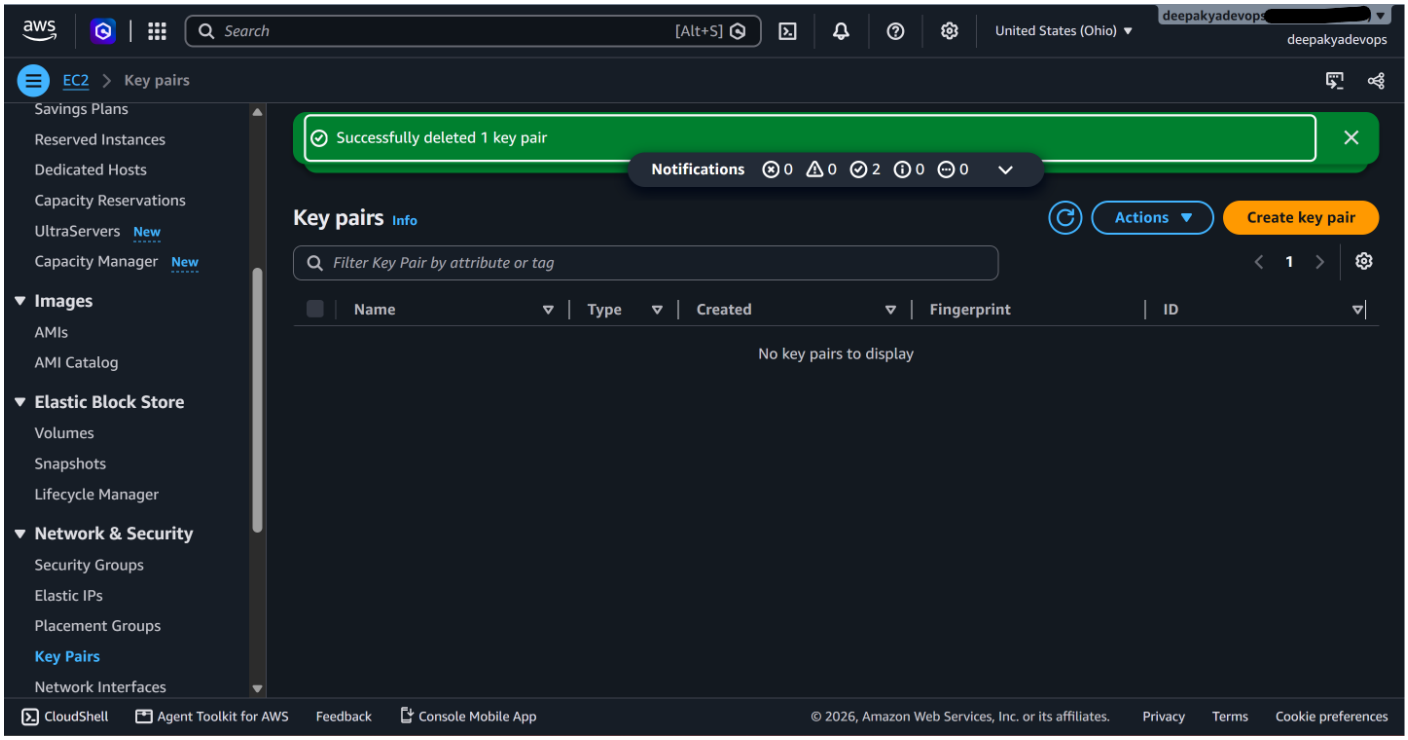


Figure 20. Cleanup, key pairs:

Figure 26. Cleanup, VPCs: only vpc-0122188984f329f5c (the default VPC) is left. Neither project VPC is listed.

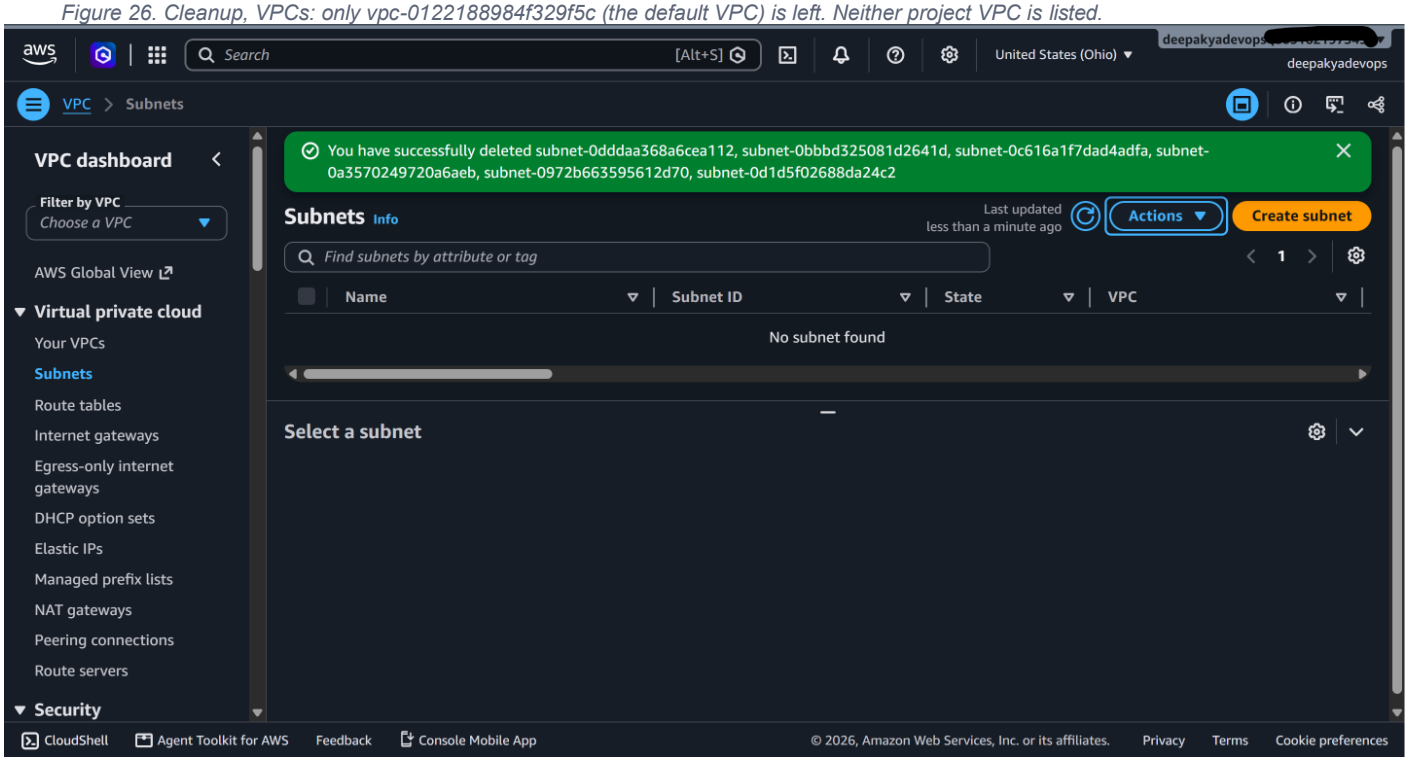


Figure 21. Cleanup, subnets

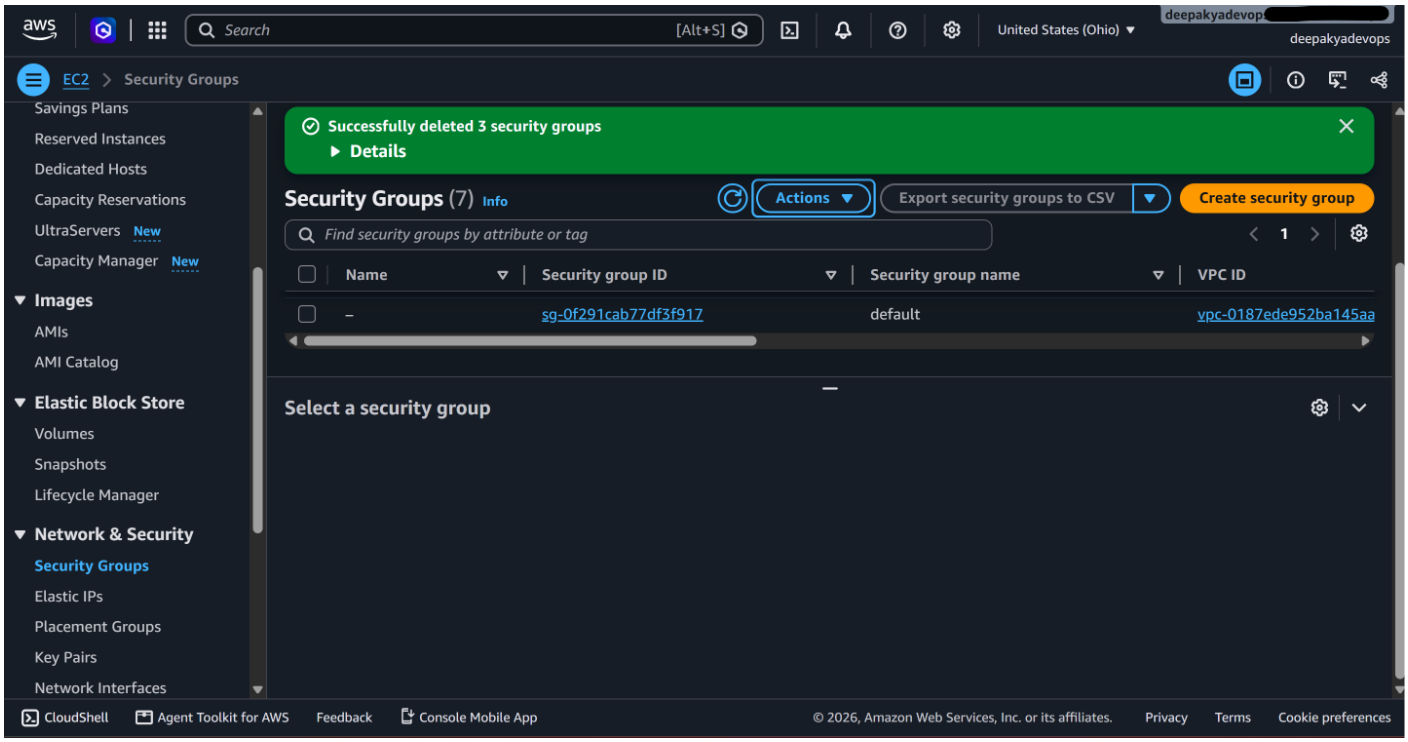


Figure 22. Cleanup, security groups.

12 / REFLECTION

What I learned and conclusion

My notes on the project.

Learning note

During this internship, I gained practical experience in AWS networking and learned how different cloud resources work together. Working on the VPC Peering project helped me understand how to connect two separate Virtual Private Clouds and enable communication between instances using their IP addresses inside the VPCs.

I also learned the importance of route tables, security groups, and network rules in controlling communication between resources. Testing connectivity through ping and SSH helped me understand how to troubleshoot network issues and identify configuration mistakes.

Overall, this project gave me a better understanding of AWS networking beyond theoretical concepts. It also taught me the importance of checking configurations carefully, testing connectivity step by step, and cleaning up resources after completing a lab. This experience has increased my confidence in working with cloud technologies and motivated me to learn more about AWS and cloud infrastructure.

Observations from my setup

- The ICMP rule on the VPC2 security group allows the whole VPC-A CIDR (28.0.0.0/16). It could be limited to the VPC-A instance's IP.
- I do not have a screenshot of the security group change around the failed ping, so I cannot say why it failed. Next time I will capture each change as I make it.

Conclusion

I connected two VPCs in us-east-2 with an Active peering connection and added routes on both sides. My setup differs from the workbook design: I used two instances, and the VPC-A instance has a public IPv4 address. Ping worked in both directions (4 of 4 replies each way), and I logged in by SSH to the instance with no public IP through the VPC-A instance. I did not capture a ping or SSH test from before peering, and I cannot say what caused the one failed ping